

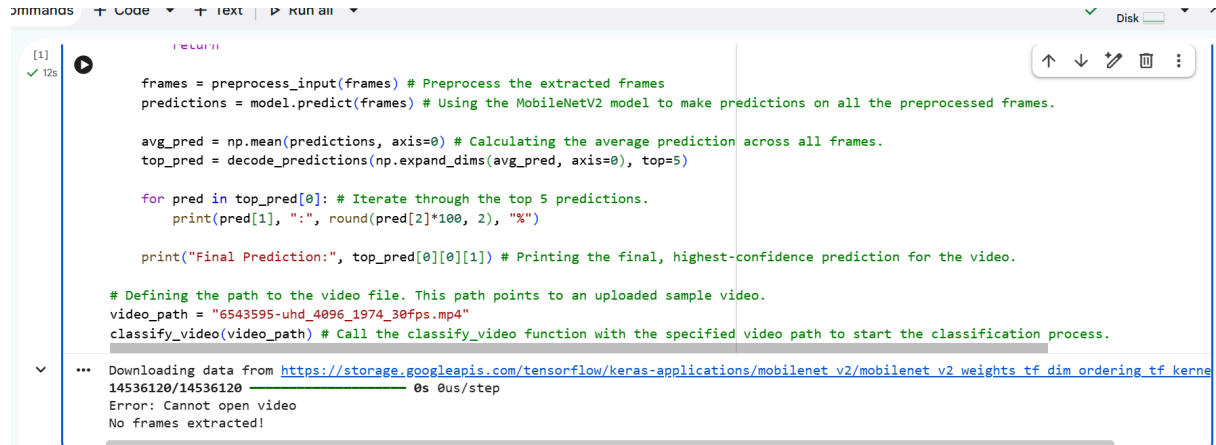
EDGE INTELLIGENCE MACSE604

19th Feb 2026 - LAB 6 Video Classification

Shruti Pandey - 25MML0020

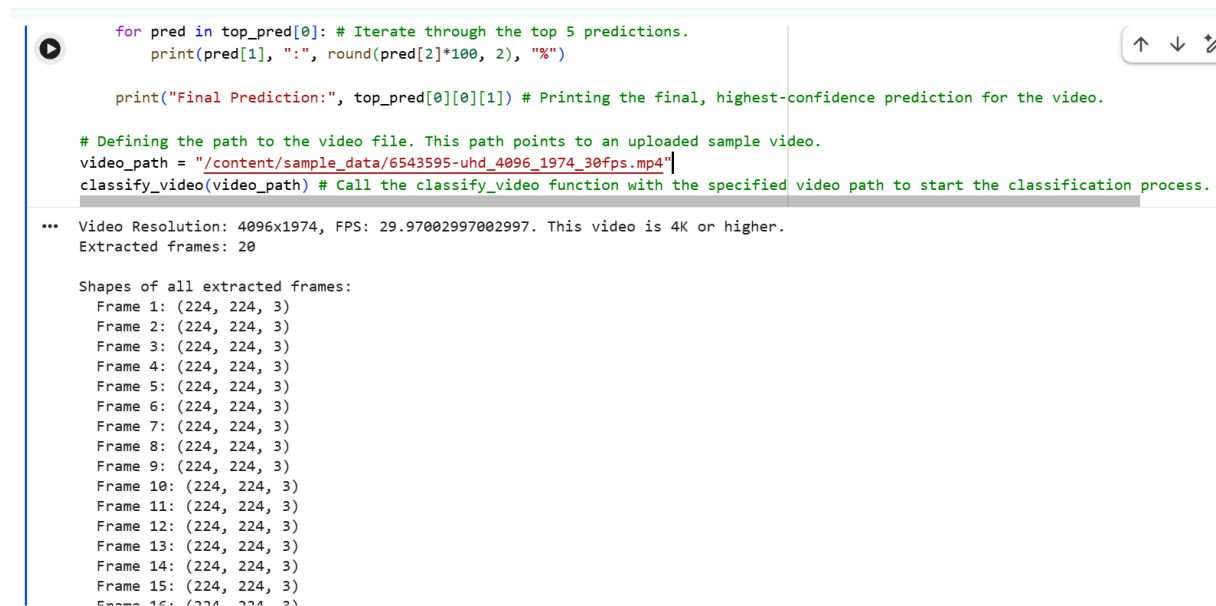
Errors encountered in the lab work :

1. Wrong path mentioned in the code



```
[1] ✓ 12s  
frames = preprocess_input(frames) # Preprocess the extracted frames  
predictions = model.predict(frames) # Using the MobileNetV2 model to make predictions on all the preprocessed frames.  
  
avg_pred = np.mean(predictions, axis=0) # Calculating the average prediction across all frames.  
top_pred = decode_predictions(np.expand_dims(avg_pred, axis=0), top=5)  
  
for pred in top_pred[0]: # Iterate through the top 5 predictions.  
    print(pred[1], ":", round(pred[2]*100, 2), "%")  
  
print("Final Prediction:", top_pred[0][0][1]) # Printing the final, highest-confidence prediction for the video.  
  
# Defining the path to the video file. This path points to an uploaded sample video.  
video_path = "6543595-uhd_4096_1974_30fps.mp4"  
classify_video(video_path) # Call the classify_video function with the specified video path to start the classification process.  
...  
Downloading data from https://storage.googleapis.com/tensorflow/keras-applications/mobilenet\_v2/mobilenet\_v2\_weights\_tf\_dim\_ordering\_tf\_kernels/14536120/14536120 0s 0us/step  
Error: Cannot open video  
No frames extracted!
```

- Corrected the error by modifying the path name to
"/content/sample_data/6543595-uhd_4096_1974_30fps.mp4"



```
for pred in top_pred[0]: # Iterate through the top 5 predictions.  
    print(pred[1], ":", round(pred[2]*100, 2), "%")  
  
print("Final Prediction:", top_pred[0][0][1]) # Printing the final, highest-confidence prediction for the video.  
  
# Defining the path to the video file. This path points to an uploaded sample video.  
video_path = "/content/sample_data/6543595-uhd_4096_1974_30fps.mp4"  
classify_video(video_path) # Call the classify_video function with the specified video path to start the classification process.  
...  
Video Resolution: 4096x1974, FPS: 29.97002997002997. This video is 4K or higher.  
Extracted frames: 20  
  
Shapes of all extracted frames:  
Frame 1: (224, 224, 3)  
Frame 2: (224, 224, 3)  
Frame 3: (224, 224, 3)  
Frame 4: (224, 224, 3)  
Frame 5: (224, 224, 3)  
Frame 6: (224, 224, 3)  
Frame 7: (224, 224, 3)  
Frame 8: (224, 224, 3)  
Frame 9: (224, 224, 3)  
Frame 10: (224, 224, 3)  
Frame 11: (224, 224, 3)  
Frame 12: (224, 224, 3)  
Frame 13: (224, 224, 3)  
Frame 14: (224, 224, 3)  
Frame 15: (224, 224, 3)  
Frame 16: (224, 224, 3)
```

2. Error while classification as the colors RGB and BGR gets swapped

```

ret, frame = cap.read() # to read frame .
if not ret: # If 'ret' is False, it means there are no more frames to read, so break the loop.
    break

if count % step == 0: # Checking if the current frame should be extracted based on the calculated step
    frame = cv2.resize(frame, (224, 224)) # Resizing the frame
    frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB) # Converting the frame
    frames.append(frame) # Adding the processed frame to the list of extracted frames.

count += 1

```

```

[2] ✓ Os
frames = preprocess_input(frames) # Preprocess the extracted frames
predictions = model.predict(frames) # Using the MobileNetV2 model to make predictions on all the preprocessed frames.

avg_pred = np.mean(predictions, axis=0) # Calculating the average prediction across all frames.
top_pred = decode_predictions(np.expand_dims(avg_pred, axis=0), top=5)

for pred in top_pred[0]: # Iterate through the top 5 predictions.
    print(pred[1], ":", round(pred[2]*100, 2), "%")

print("Final Prediction:", top_pred[0][0][1]) # Printing the final, highest-confidence prediction for the video.

# Defining the path to the video file. This path points to an uploaded sample video.
video_path = "/content/sample_data/6543595-uhd_4096_1974_30fps.mp4"
classify_video(video_path) # Call the classify_video function with the specified video path to start the classification process.

... Error: Cannot open video
No frames extracted!

```

- Corrected the error by converting frames from BGR to RGB format so they match the color format “ `frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)` ”

3. Indentation Error

```

[3] ✓ Os
print(f" Frame {i+1}: {frame.shape}") # Printing the shape of each extracted frame.
return np.array(frames)

def classify_video(video_path):
    # This function classifies the content of a video by processing its frames through a pre-trained image classification model.
    # video_path: The file path to the input video.

    frames = extract_frames(video_path) # Call the extract_frames function to get frames from the video.

    if len(frames) == 0: # Check if no frames were extracted.
        print("No frames extracted!") # Print a message if no frames are available for classification.
        return

    frames = preprocess_input(frames) # Preprocess the extracted frames
    predictions = model.predict(frames) # Using the MobileNetV2 model to make predictions on all the preprocessed frames.

    avg_pred = np.mean(predictions, axis=0) # Calculating the average prediction across all frames.
    top_pred = decode_predictions(np.expand_dims(avg_pred, axis=0), top=5)

    for pred in top_pred[0]: # Iterate through the top 5 predictions.
        print(pred[1], ":", round(pred[2]*100, 2), "%")

    print("Final Prediction:", top_pred[0][0][1]) # Printing the final, highest-confidence prediction for the video.

# Defining the path to the video file. This path points to an uploaded sample video.
video_path = "/content/sample_data/6543595-uhd_4096_1974_30fps.mp4"
classifyv video(video path) # Call the classifyv video function with the specified video path to start the classification process.

```

```
[3] 0s
avg_pred = np.mean(predictions, axis=0) # Calculating the average prediction across all frames.
top_pred = decode_predictions(np.expand_dims(avg_pred, axis=0), top=5)

for pred in top_pred[0]: # Iterate through the top 5 predictions.
    print(pred[1], ":", round(pred[2]*100, 2), "%")

print("Final Prediction:", top_pred[0][0][1]) # Printing the final, highest-confidence prediction for the video.

# Defining the path to the video file. This path points to an uploaded sample video.
video_path = "/content/sample_data/6543595-uhd_4096_1974_30fps.mp4"
classify_video(video_path) # Call the classify_video function with the specified video path to start the classification process.

... File "/tmp/ipython-input-191954366.py", line 77
      return
      ^
SyntaxError: 'return' outside function
```

- Corrected by rightly aligning the loop in the code